I-AM-IOS: Specification

Intelligent Autonomous Modular Integrated Operating System

Version 2.0 | Status: Production

Table of Contents

1. Executive Summary
2. Core Philosophy
3. Architecture Overview
4. Layer Specifications
5. Data Model
6. Consensus Protocol
7. Determinism Guarantees
8. API Reference
9. Deployment Guide
10. Appendices

Executive Summary

I-AM-IOS is a browser-native, event-sourced distributed operating system that computes application state deterministically from an immutable event log rather than storing mutable state in a database.

Core invariant:

```
VM_state(n) = deriveState(eventLog[0..n])
```

Key properties:

- ✅ 100% deterministic state derivation
- ✅ Time-machine replay to any historical point
- ✅ Cryptographically verified audit trail (hash chain)

- ✅ Byzantine fault tolerance (>2/3 quorum finality)
- ✅ Offline-first operation (all logic client-side)
- ✅ Zero server infrastructure required
- ✅ P2P consensus via WebRTC mesh
- ✅ Sequential throughput (1 event/block), unlimited latency tolerance

Market fit: Collaborative tools, DAOs, governance, knowledge work, privacy-first applications.

Not suitable for: High-frequency trading, real-time media, legacy system integration.

Core Philosophy

Why I-AM-IOS Exists

Existing systems force a choice:

1. **Centralized (AWS, Vercel, Google Cloud)**
    - Fast, reliable
    - Single point of failure
    - Vendor lock-in
    - Non-deterministic (server timing)
    - No offline operation
2. **Decentralized (Blockchain, IPFS)**
    - Trustless consensus
    - Slow (10+ seconds finality)
    - Complex (50K+ LOC)
    - High bandwidth overhead
    - Environmental cost (PoW)
3. **Local (Redux, Zustand, IndexedDB)**
    - Fast, deterministic
    - No persistence
    - No multiuser sync
    - No auditability

    - Manual conflict resolution

I-AM-IOS unifies all three:

- **Deterministic** like local state management
- **Decentralized** like blockchain (no server)
- **Auditable** like event sourcing
- **Fast** like Redis (browser-native)
- **Simple** (5K LOC kernel vs 50K+ competitors)

Design Principles

1. **Event Log is Truth**
    - State is derived, not stored
    - Audit trail is not separate—it's the foundation
    - No data duplication
2. **Pure Functions Only**
    - `deriveState(eventLog)` has no side effects
    - No timestamps, randomness, or I/O in execution path
    - Consequence: deterministic convergence across all peers
3. **Immutable Records**
    - Events frozen at creation
    - Hash chain proves integrity
    - Tamper detection automatic
4. **Browser-First**
    - No server required
    - All logic runs client-side
    - P2P sync is optional (works offline indefinitely)
5. **Cryptographic Proof**
    - Hash chain makes corruption impossible to hide
    - Suitable for compliance (financial, healthcare, legal)

Architecture Overview

7-Layer Stack

```
┌──────────────────────────────────────────────────────────────┐
│ L1: Surfaces (User Interfaces)                                │
│     5 HTML apps, cross-browser, deterministic rendering       │
├──────────────────────────────────────────────────────────────┤
│ L2: Sovereign Bus (Cross-Tab Sync)                            │
│     BroadcastChannel('sovereign-os-bus'), synchronous emit    │
├──────────────────────────────────────────────────────────────┤
│ L3: Sovereign Log (Truth Engine)                              │
│     Event sourcing, time-machine state derivation, IndexedDB│
├──────────────────────────────────────────────────────────────┤
│ L4: Sovereign Network (P2P Consensus)                         │
│     WebRTC mesh, gossip protocol, >2/3 quorum finality        │
├──────────────────────────────────────────────────────────────┤
│ L5: Rekernel (Deterministic Kernel)                           │
│     6 security locks, canonical ordering, transition chain    │
├──────────────────────────────────────────────────────────────┤
│ L6: UDCSEF Fabric (Distributed Compute)                       │
│     JSONFlow IR, pure operations, deterministic execution     │
├──────────────────────────────────────────────────────────────┤
│ L7: WASM Kernel (Optimized Runtime)                           │
│     Rust, WebAssembly, FFI, 100x faster crypto                │
└──────────────────────────────────────────────────────────────┘
```

Information Flow

```
L1: User action (click, type, command)
  ↓
L2: emit(event) → BroadcastChannel (all tabs receive)
  ↓
L3: deriveState(eventLog) → pure function → new state
  ↓
L4: Promote to network → gossip to peers → accumulate >2/3 acks
  ↓
L5: Verify (I1-I6) → Order (sort by hash) → Execute → Transition
  ↓
```

```
L6: If compute job → JSONFlow IR → deterministic ops
  ↓
L7: If crypto-heavy → delegate to WASM → 100x faster
  ↓
Finality: Event final after k+1 blocks (default k=1 → 2 blocks)
```

Layer Specifications

L1: Surfaces (User Interfaces)

Purpose: Provide rich interactive applications that emit events to the system.

Components:

1. Portal Dashboard (`index.html`)

- Live event stream visualization
- Ledger statistics and height
- Network peer status
- Launch pad for other surfaces
- Real-time metrics

Features:

- Event log viewer (searchable, filterable)
- State snapshots (current + historical)
- Memory stats (conversation count, message count)
- Peer list (connected nodes, ping times)
- Network health (consensus strength, lag)

2. App Builder (`app-builder-v2.html`)

- Natural language → JSONFlow IR → executable code
- Three-stage pipeline
- Visual debugging
- Module versioning

Capabilities:

- NL input: "Create a sentiment analyzer"
- Stage 1: Parse intent
- Stage 2: Generate JSONFlow IR
- Stage 3: Emit code artifact
- Deploy to peers

3. Attack Command (`attack.html`)

- Security audit and vulnerability scanner
- Smart contract analysis
- Consensus verification
- Byzantine behavior detection

Features:

- Load program (JSONFlow or bytecode)
- Analyze for safety violations
- Simulate Byzantine attacks
- Report vulnerabilities
- Recommend fixes

4. UDCSEF Fabric (`generate-value.html`)

- P2P compute execution
- Real-time job status
- Result streaming
- Network load visualization

Capabilities:

- Submit compute job (JSONFlow IR)
- Monitor execution across peers
- Stream results as they arrive

- Verify deterministic consistency
- Export results

5. Genesis (`apps/index1.html`)

- 3-stage compilation pipeline
- Flow language to JavaScript
- Constraint validation
- Bytecode generation

Stages:

1. **Parsing:** Lex/parse flow syntax → AST
2. **Analysis:** Type checking, constraint verification, safety checks
3. **Emission:** Codegen (JS, JSONFlow IR, WASM)

Technologies: HTML5, BroadcastChannel API, IndexedDB, PeerJS

Constraints:

- All rendering must be deterministic (no animation timing deps)
- No non-deterministic APIs (Math.random, Date.now in rendering)
- Event emission only (no direct state mutation)

L2: Sovereign Bus

Purpose: Real-time cross-tab synchronization using browser-native APIs.

Mechanism: BroadcastChannel('sovereign-os-bus')

```javascript
```

```
// Broadcast (L1 → L2)
sovereignLog.emit({
  type: 'MODEL_SELECTED',
  model: 'llama3'
});
// ↓ BroadcastChannel sends to all tabs

// Receive (L2 → subscribers)
channel.onmessage = (event) => {
  const { record, state } = event.data;
  // Subscribers notified synchronously
  callbacks.forEach(fn => fn(state, record));
};
```

Key properties:

- **Synchronous:** No event loop pollution
- **Same-origin only:** Security boundary
- **All-to-all broadcast:** No star topology
- **No persistence:** Transient, in-memory

Event Schema:

```typescript
interface BusEvent {
  record: {
    type: string;
    seq: number;
    ts: number;
    hash: string;
    prevHash: string;
    [key: string]: any;
  };
  state: AppState;  // Derived state after event
}
```

Latency: <1ms (same process, no network)

Throughput: Unlimited (in-memory, no I/O)

Error handling:

- If subscriber throws, exception caught and logged
- Other subscribers notified normally
- Bus continues operating

L3: Sovereign Log (Truth Engine)

Purpose: Event sourcing with time-machine capabilities and cryptographic integrity.

Core equation:

```
VM_state(n) = deriveState(eventLog[0..n])
```

Event Structure:

```typescript
interface EventRecord {
  // Identity
  seq: number;              // Sequence number (0, 1, 2, ...)
  type: string;             // Event type (must be in EVENT_TYPES)

  // Payload
  [key: string]: any;       // Event-specific data

  // Metadata
  ts: number;               // Timestamp (Date.now())

  // Cryptographic proof
  hash: string;             // FNV-32(payload | prevHash)
  prevHash: string;         // Hash of previous event

  // Frozen
  __frozen: true;           // Object.freeze() applied
}
```

Hashing Algorithm: FNV-1a 32-bit

```typescript
function fnv32(str: string): string {
  let h = 2166136261;  // FNV offset basis
  for (let i = 0; i < str.length; i++) {
    h ^= str.charCodeAt(i);
    h = Math.imul(h, 16777619);  // FNV prime
  }
  return (h >>> 0).toString(16).padStart(8, '0');
}

function hashEvent(payload: any, prevHash: string): string {
  const payloadStr = JSON.stringify(payload);
  return fnv32(payloadStr + '|' + prevHash);
}
```

Why FNV-32?

- Speed: O(n) single pass, no crypto overhead
- Browser-safe: No BigInt, no external libraries
- Sufficient for deduplication (collisions rare)
- Not cryptographically secure (OK, not used for security)

Pure Derivation Function:

```typescript
```

```
function deriveState(eventLog: EventRecord[]): AppState {
  let state: AppState = {
    model: null,
    messages: [],
    kernelMode: false,
    memoryCount: 0,
    // ... other fields
  };

  for (const event of eventLog) {
    switch (event.type) {
      case 'MODEL_SELECTED':
        state.model = event.model;
        break;
      case 'MESSAGE_ADDED':
        state.messages.push(event.message);
        break;
      case 'KERNEL_MODE_TOGGLED':
        state.kernelMode = event.enabled;
        break;
      // ... other cases
      default:
        throw new Error(`Unknown event type: ${event.type}`);
    }
  }

  return state;
}
```

Guarantee: `deriveState` must be a **pure function**:

- ❌ No Date.now() (except in input)
- ❌ No Math.random()
- ❌ No I/O (fetch, file system)
- ❌ No mutation of shared state
- ❌ No side effects
- ✅ Same input → same output (always)

Time-Machine Capabilities:

```typescript
// Current state
const now = deriveState(eventLog);

// Historical state (n events ago)
const past = deriveState(eventLog.slice(0, eventLog.length - n));

// State at specific sequence
const atSeq = deriveState(eventLog.slice(0, targetSeq + 1));

// State at specific timestamp
const atTime = deriveState(
  eventLog.filter(e => e.ts <= targetTimestamp)
);
```

Persistence: IndexedDB

```typescript
// Save (automatic on every emit)
await IndexedDB.setItem('sovereign-log', JSON.stringify(eventLog));

// Load (on app startup)
const savedLog = await IndexedDB.getItem('sovereign-log');
const verified = replay(JSON.parse(savedLog));
const state = deriveState(verified);
```

Replay & Verification:

```typescript
```

```typescript
function replay(savedLog: EventRecord[]): EventRecord[] {
  let prevHash = '0000000000000000';

  for (const record of savedLog) {
    const { hash, prevHash: recordPrevHash, ...payload } = record;
    const expected = hashEvent(payload, prevHash);

    if (hash !== expected) {
      throw new Error(
        `[Integrity failure at seq ${record.seq}] ` +
        `Expected ${expected}, got ${hash}. Log may be tampered.`
      );
    }

    prevHash = hash;
  }

  return savedLog;
}
```

Snapshots (Optional Optimization):

```typescript
```

```typescript
interface Snapshot {
  height: number;           // Event count
  timestamp: number;        // When created
  state: AppState;          // Derived state at height
  stateHash: string;        // hash(state) for verification
  eventLog: EventRecord[];  // Full log up to height
}

// Create every 1000 events
if (eventLog.length % 1000 === 0) {
  const snapshot: Snapshot = {
    height: eventLog.length,
    timestamp: Date.now(),
    state: deriveState(eventLog),
    stateHash: fnv32(JSON.stringify(deriveState(eventLog))),
    eventLog: [...eventLog]
  };
  await IndexedDB.setItem(`snapshot-${snapshot.height}`, snapshot);
}

// Recovery: Load snapshot + replay suffix
const snapshot = await IndexedDB.getItem('snapshot-1000');
const suffix = eventLog.slice(1000);
const state = deriveState([...snapshot.eventLog, ...suffix]);
```

Event Registry:

```typescript
```

```
enum EVENT_TYPES {
  // UI
  MODEL_SELECTED = 'MODEL_SELECTED',
  TAB_CHANGED = 'TAB_CHANGED',

  // Network
  STREAMING_STARTED = 'STREAMING_STARTED',
  STREAMING_ENDED = 'STREAMING_ENDED',
  OLLAMA_STATUS = 'OLLAMA_STATUS',

  // Kernel
  KERNEL_MODE_TOGGLED = 'KERNEL_MODE_TOGGLED',
  KERNEL_STARTED = 'KERNEL_STARTED',
  KERNEL_VIEW_RESOLVED = 'KERNEL_VIEW_RESOLVED',
  KERNEL_ANALYSIS = 'KERNEL_ANALYSIS',
  KERNEL_ERROR = 'KERNEL_ERROR',

  // Compute
  FLOW_STAGE_ENTERED = 'FLOW_STAGE_ENTERED',
  FLOW_MODULE_DEFINED = 'FLOW_MODULE_DEFINED',
  FLOW_COMPILED = 'FLOW_COMPILED',
  FLOW_CODE_EMITTED = 'FLOW_CODE_EMITTED',

  // Memory
  MEMORY_IMPORTED = 'MEMORY_IMPORTED',
  MEMORY_MERGED = 'MEMORY_MERGED',
  MEMORY_CLEARED = 'MEMORY_CLEARED',

  // System
  SNAPSHOT = 'SNAPSHOT'
}
```

API:

```typescript
```

```
export function emit(event: Partial<EventRecord>): EventRecord;
export function deriveState(log?: EventRecord[]): AppState;
export function snapshot(): { log: EventRecord[]; head: string; seq: number };
export function replay(savedLog: EventRecord[]): EventRecord[];
export function restore(savedLog: EventRecord[]): AppState;
export function getLog(): EventRecord[];
export function subscribe(callback: (state: AppState, record: EventRecord) => void)
```

Constraints:

- Events must be JSON-serializable
- All event payloads immutable
- Unknown event types throw error (fail-fast)
- deriveState must be deterministic
- No async operations in emit or derive

L4: Sovereign Network (P2P Consensus)

Purpose: Distributed ledger persistence and finality with Byzantine tolerance.

Topology: WebRTC peer-to-peer mesh

```
Peer A ↔ Peer B
  ↑         ↓
  └↔ Peer C
```

Mechanism:

1. **Peer Discovery (PeerJS)**
    - Each peer connects to bootstrap node (or DHT)
    - PeerJS establishes WebRTC data channels
    - Mesh form naturally (all-to-all connectivity)
2. **Event Gossip**
    - When event reaches quorum locally, promote to network
    - Broadcast event to all connected peers

- Each peer stores in IndexedDB ledger
- Duplicate detection by hash

3. **Acknowledgment Protocol**

```
Peer A: "I have event E (hash=ABC123)"
  ↓ (gossip to B, C, D)
Peer B: ack={validatorId: B, eventHash: ABC123, sig: ...}
Peer C: ack={validatorId: C, eventHash: ABC123, sig: ...}
Peer D: ack={validatorId: D, eventHash: ABC123, sig: ...}
  ↓ (acks gossipped back)
Quorum check: acks >= ceil(2N/3 + 1) → event final
```

4. **Finality Rule**

```
event_final IFF acks_received >= ceil(2N/3 + 1)

For N peers:
- N=3: need 3 acks (100% must agree)
- N=4: need 3 acks (75%)
- N=5: need 4 acks (80%)
- N=7: need 5 acks (71%)

Byzantine tolerance: can tolerate up to floor(N/3) faulty nodes
```

Data Structures:

```typescript
```

```typescript
interface PeerInfo {
  peerId: string;
  pubkey: string;
  lastSeen: number;
  reputation: number;  // 0-100, decreased on misbehavior
}

interface LedgerEntry {
  type: 'ACCEPTED' | 'REJECTED' | 'SNAPSHOT';
  event: EventRecord;
  transitions: TransitionRecord[];
  timestamp: number;
}

interface Acknowledgment {
  validatorId: string;
  eventHash: string;
  timestamp: number;
  signature: string;  // Signed by validator's private key
}
```

Persistence: IndexedDB

```typescript
// Each peer maintains own copy
const ledger: LedgerEntry[] = await IndexedDB.getItem('sovereign-ledger');

// Append on acceptance
await IndexedDB.appendItem('sovereign-ledger', {
  type: 'ACCEPTED',
  event: verifiedEvent,
  transitions: [transitionRecord],
  timestamp: Date.now()
});
```

Gossip Protocol:

```typescript
```

```
// When event is locally verified
sovereignLog.emit(event);

// L3 notifies L4
onLocalStateChange((state, record) => {
  // Promote to network
  network.broadcastEvent(record);
});

// Peers receive
peer.on('data', (msg) => {
  if (msg.type === 'EVENT_BROADCAST') {
    const event = msg.event;

    // Verify (I1-I6 checks)
    const result = verifyEvent(event);
    if (!result.valid) {
      return;  // Ignore invalid events
    }

    // Store locally
    await IndexedDB.appendItem('sovereign-ledger', {
      type: 'ACCEPTED',
      event,
      transitions: []
    });

    // Acknowledge
    peer.send({
      type: 'ACK',
      eventHash: event.hash,
      validatorId: myPeerId,
      signature: sign(event.hash, myPrivateKey)
    });
  }
});
```

Byzantine Safety:

- **Problem:** Two peers might have different ledgers

- **Solution:** Quorum required (>2/3 must agree)
- **Consequence:** No fork possible if <1/3 Byzantine
- **Proof:** If two conflicting events both finalize, they must overlap in validators. Overlapping validator signed both → slashing.

API:

```typescript
export function addPeer(peerId: string, pubkey: string): void;
export function broadcastEvent(event: EventRecord): void;
export function getAcknowledgments(eventHash: string): Acknowledgment[];
export function isEventFinal(eventHash: string): boolean;
export function getLedger(): LedgerEntry[];
export function getPeers(): PeerInfo[];
```

Constraints:

- All messages signed (except discovery)
- Duplicate events deduplicated by hash
- Peer reputation tracked (bad actors downweighted)
- Network partition → events accumulate, finalize on reconnect

---

L5: Rekernel (Deterministic Kernel)

Purpose: Byzantine fault tolerance with deterministic ordering and tamper-evident ledger.

The 6 Security Locks:

S1: Protocol Freeze

```typescript
```

```typescript
const HASH_PROTOCOL_VERSION: number = 1;
const ACCEPTED_VERSIONS: Set<number> = new Set([1]);

function verifyProtocolVersion(event: EventRecord): boolean {
  return ACCEPTED_VERSIONS.has(event.protocolVersion ?? 1);
}
```

Guarantee: Version mismatch causes explicit fork (DivergeRecord), not silent incompatibility.

S2: Mandatory Ingress Verification (I1–I6)

```typescript
```

```typescript
interface VerificationResult {
  valid: boolean;
  violations: string[];
}

function verifyEvent(event: EventRecord): VerificationResult {
  const violations: string[] = [];

  // I1: Hash consistency
  if (hashEvent(event.payload, event.prevHash) !== event.hash) {
    violations.push(`HASH_MISMATCH`);
  }

  // I2: ID derivation
  const derivedId = deriveEventId(event);
  if (event.id !== derivedId) {
    violations.push(`ID_MISMATCH`);
  }

  // I3: Protocol version
  if (!ACCEPTED_VERSIONS.has(event.protocolVersion ?? 1)) {
    violations.push(`VERSION_MISMATCH`);
  }

  // I4: Required fields
  const requiredFields = ['type', 'seq', 'ts', 'hash', 'prevHash'];
  for (const field of requiredFields) {
    if (!(field in event)) {
      violations.push(`MISSING_FIELD: ${field}`);
    }
  }

  // I5: Clock skew
  const now = Date.now();
  const skew = Math.abs(now - event.ts);
  if (skew > CLOCK_SKEW_TOLERANCE) {
    violations.push(`CLOCK_SKEW: ${skew}ms`);
  }

  // I6: JSON serializable
```

```
  try {
    JSON.stringify(event.payload);
  } catch (e) {
    violations.push(`NOT_SERIALIZABLE: ${e.message}`);
  }

  return {
    valid: violations.length === 0,
    violations
  };
}
```

Constants:

```typescript
const CLOCK_SKEW_TOLERANCE = 60_000;  // 60 seconds
```

S3: Rejection Records

```typescript
```

```typescript
interface RejectionRecord {
  type: 'REJECTION';
  rejectedHash: string;
  rejectedId: string;
  reason: RejectionReason;
  violations: string[];
  stateHash: string;
  prevHash: string;
  timestamp: number;
}

enum RejectionReason {
  HASH_MISMATCH = 'HASH_MISMATCH',
  ID_MISMATCH = 'ID_MISMATCH',
  VERSION_MISMATCH = 'VERSION_MISMATCH',
  MISSING_FIELDS = 'MISSING_FIELDS',
  CLOCK_SKEW = 'CLOCK_SKEW',
  NOT_SERIALIZABLE = 'NOT_SERIALIZABLE',
  DUPLICATE = 'DUPLICATE'
}

// Failed events → ledger entries (not discarded)
if (!verifyResult.valid) {
  const rejection = createRejectionRecord({
    rejectedHash: event.hash,
    rejectedId: event.id,
    reason: verifyResult.violations[0],
    violations: verifyResult.violations,
    stateHash: currentStateHash,
    prevHash: lastTransitionHash
  });

  ledger.push(rejection);
  return { accepted: false, rejection };
}
```

S4: Event Ordering (Deterministic Total Order)

typescript

```typescript
function canonicalizeEventBatch(events: EventRecord[]): EventRecord[] {
  // Deduplicate by hash
  const deduped = new Map<string, EventRecord>();
  for (const e of events) {
    deduped.set(e.hash, e);
  }

  // Sort deterministically
  return Array.from(deduped.values()).sort((a, b) => {
    // Primary: hash lexicographic order
    const hashCmp = a.hash.localeCompare(b.hash);
    if (hashCmp !== 0) return hashCmp;

    // Secondary: timestamp (tie-break)
    const tsCmp = a.ts - b.ts;
    if (tsCmp !== 0) return tsCmp;

    // Tertiary: id (final tie-break)
    return a.id.localeCompare(b.id);
  });
}
```

Guarantee: All nodes compute identical order for same event set. No consensus needed for ordering.

S5: Transition Chain (Tamper-Evident)

```typescript
```

```typescript
interface TransitionRecord {
  index: number;                    // Block height
  prevTransitionHash: string;       // Link to previous
  event: EventRecord;               // The event that caused it
  preStateHash: string;             // State before execution
  postStateHash: string;            // State after execution
  transitionHash: string;           // T_i = hash(T_{i-1}, E_i, S_i)
  timestamp: number;
}

async function createTransitionRecord(opts: {
  index: number;
  prevTransitionHash: string;
  event: EventRecord;
  preStateHash: string;
  postStateHash: string;
}): Promise<TransitionRecord> {
  const hashInput = {
    index: opts.index,
    prevTransitionHash: opts.prevTransitionHash,
    eventHash: opts.event.hash,
    preStateHash: opts.preStateHash,
    postStateHash: opts.postStateHash
  };

  const transitionHash = await sha256(JSON.stringify(hashInput));

  return {
    index: opts.index,
    prevTransitionHash: opts.prevTransitionHash,
    event: opts.event,
    preStateHash: opts.preStateHash,
    postStateHash: opts.postStateHash,
    transitionHash,
    timestamp: Date.now()
  };
}
```

Property: Altering any T_i invalidates all downstream hashes.

```
T_0 = hash(genesis, E_0, S_0)
T_1 = hash(T_0, E_1, S_1)
T_2 = hash(T_1, E_2, S_2)
...

If T_1 is modified, hash(T_2) no longer matches → corruption detected.
```

S6: Snapshots (Periodic Checkpoints)

```typescript
interface Snapshot {
  height: number;
  timestamp: number;
  state: AppState;
  stateHash: string;
  snapshotHash: string;
}

const SNAPSHOT_INTERVAL = 1000;  // Every 1000 events

if (kernelHeight % SNAPSHOT_INTERVAL === 0) {
  const snapshot: Snapshot = {
    height: kernelHeight,
    timestamp: Date.now(),
    state: currentState,
    stateHash: await sha256(JSON.stringify(currentState)),
    snapshotHash: await sha256(
      JSON.stringify({
        height: kernelHeight,
        timestamp: Date.now(),
        stateHash: hashValue
      })
    )
  };

  await IndexedDB.setItem(`snapshot-${snapshot.height}`, snapshot);
}
```

Recovery:

```typescript
// Load snapshot
const snapshot = await IndexedDB.getItem('snapshot-1000');

// Verify
if (!verifySnapshotHash(snapshot)) {
  throw new Error('Snapshot corrupted');
}

// Replay suffix
const suffix = transitionChain.slice(1000);
const finalState = deriveStateFromTransitions(
  snapshot.state,
  suffix
);
```

API:

```typescript
export function verifyEvent(event: EventRecord): VerificationResult;
export function canonicalizeEventBatch(events: EventRecord[]): EventRecord[];
export function createTransitionRecord(opts: any): Promise<TransitionRecord>;
export function verifyTransitionChain(chain: TransitionRecord[]): boolean;
export function createSnapshot(state: AppState, height: number): Promise<Snapshot>;
export function verifySnapshot(snapshot: Snapshot): boolean;
```

Constraints:

- All event inputs validated before execution
- All state transitions chained together
- No event can be silently dropped
- No ordering ambiguity
- Replaying from genesis always produces same final state

L6: UDCSEF Fabric (Distributed Compute)

Purpose: Deterministic, decentralized computation across P2P network.

JSONFlow IR Specification:

```typescript
interface JSONFlowModule {
  module_name: string;
  version: string;
  description?: string;

  inputs: Parameter[];
  outputs: Parameter[];
  steps: Step[];
  constraints?: string[];
}

interface Parameter {
  name: string;
  type: 'string' | 'number' | 'boolean' | 'array' | 'object';
  description?: string;
  default?: any;
}

interface Step {
  id: string;                           // Unique within module
  op: string;                           // Operation name
  inputs: Record<string, Reference>;    // Input bindings
  outputs: Record<string, Reference>;   // Output bindings
  config?: Record<string, any>;         // Op-specific config
}

type Reference = string | number | boolean | null;  // $varName or literal
```

Example:

```json
```

```
{
  "module_name": "sentiment_analysis",
  "version": "1.0.0",
  "description": "Analyze sentiment of text",
  "inputs": [
    { "name": "text", "type": "string" }
  ],
  "outputs": [
    { "name": "sentiment", "type": "string" },
    { "name": "score", "type": "number" }
  ],
  "steps": [
    {
      "id": "tokenize",
      "op": "string_split",
      "inputs": { "text": "$text", "sep": " " },
      "outputs": { "result": "$tokens" }
    },
    {
      "id": "analyze",
      "op": "sentiment_detect",
      "inputs": { "tokens": "$tokens" },
      "outputs": { "sentiment": "$sentiment", "score": "$score" }
    }
  ],
  "constraints": [
    "text length < 10000",
    "score between 0 and 1"
  ]
}
```

Content Addressing:

```typescript
```

```typescript
function programHash(ir: JSONFlowModule): string {
  const canonical = {
    module_name: ir.module_name,
    version: ir.version,
    inputs: ir.inputs,
    outputs: ir.outputs,
    steps: ir.steps
  };

  return fnv32(JSON.stringify(canonical));
}

// Example:
// SHA256(IR) = abc123def456...
// Program cached as "abc123def456"
// Prevents redundant execution
```

Deterministic Executor:

```typescript
```

```
class JSONFlowExecutor {
  private ops: Map<string, (inputs: any) => any>;

  registerOp(name: string, fn: (inputs: any) => any): void {
    // Validate fn is pure
    this.ops.set(name, fn);
  }

  execute(ir: JSONFlowModule, inputs: Record<string, any>): {
    outputs: Record<string, any>;
    trace: ExecutionTrace[];
    success: boolean;
    error?: string;
  } {
    const env = { ...inputs };
    const trace: ExecutionTrace[] = [];

    for (const step of ir.steps) {
      const fn = this.ops.get(step.op);
      if (!fn) {
        return {
          outputs: null,
          trace,
          success: false,
          error: `Unknown op: ${step.op}`
        };
      }

      // Gather inputs
      const stepInputs: Record<string, any> = {};
      for (const [key, ref] of Object.entries(step.inputs)) {
        stepInputs[key] = typeof ref === 'string' && ref.startsWith('$')
          ? env[ref.slice(1)]
          : ref;
      }

      // Execute
      try {
        const result = fn(stepInputs);
```

```
        // Write outputs
        for (const [key, ref] of Object.entries(step.outputs)) {
          const outputVar = typeof ref === 'string' && ref.startsWith('$')
            ? ref.slice(1)
            : key;
          env[outputVar] = result[key] ?? result;
        }

        trace.push({
          stepId: step.id,
          op: step.op,
          inputs: stepInputs,
          outputs: { [Object.keys(step.outputs)[0]]: env[Object.values(step.outputs
          success: true
        });
      } catch (e) {
        return {
          outputs: null,
          trace,
          success: false,
          error: `Step ${step.id}: ${e.message}`
        };
      }
    }

    // Extract outputs
    const outputs: Record<string, any> = {};
    for (const param of ir.outputs) {
      outputs[param.name] = env[param.name];
    }

    return { outputs, trace, success: true };
  }
}
```

Built-in Operations:

```typescript
```

```
executor.registerOp('add', ({ a, b }) => a + b);
executor.registerOp('subtract', ({ a, b }) => a - b);
executor.registerOp('multiply', ({ a, b }) => a * b);
executor.registerOp('divide', ({ a, b }) => b !== 0 ? a / b : null);

executor.registerOp('string_split', ({ text, sep }) =>
  text.split(sep)
);
executor.registerOp('string_join', ({ items, sep }) =>
  items.join(sep)
);
executor.registerOp('string_length', ({ text }) =>
  text.length
);

executor.registerOp('array_length', ({ array }) =>
  array.length
);
executor.registerOp('array_map', ({ array, transform }) =>
  array.map(x => transform(x))
);

executor.registerOp('if_else', ({ condition, trueBranch, falseBranch }) =>
  condition ? trueBranch : falseBranch
);

executor.registerOp('and', ({ a, b }) => a && b);
executor.registerOp('or', ({ a, b }) => a || b);
executor.registerOp('not', ({ a }) => !a);
```

Determinism Guarantee:

```typescript
```

```
// Same IR + same inputs → same outputs (always)

const ir = { ... };
const inputs = { text: 'hello' };

const result1 = executor.execute(ir, inputs);
const result2 = executor.execute(ir, inputs);

console.assert(
  JSON.stringify(result1.outputs) === JSON.stringify(result2.outputs)
);  // Always passes
```

API:

```typescript
export class JSONFlowExecutor {
  registerOp(name: string, fn: (inputs: any) => any): void;
  execute(ir: JSONFlowModule, inputs: Record<string, any>): ExecutionResult;
}

export function programHash(ir: JSONFlowModule): string;
export function validateModule(ir: JSONFlowModule): ValidationError[];
```

Constraints:

- All ops must be pure (no side effects)
- No I/O (fetch, file system)
- No non-deterministic operations (Math.random, Date.now)
- No external state access
- All inputs/outputs JSON-serializable

L7: WASM Kernel (Optimized Runtime)

Purpose: High-performance cryptography and consensus logic via WebAssembly.

Language: Rust

Build:

```bash
cd wasm-kernel
cargo build --release --target wasm32-unknown-unknown
```

Modules:

Boundary (`boundary/mod.rs`)

- FFI: JavaScript ↔ WASM marshalling
- Type conversion
- Memory management

```rust
#[wasm_bindgen]
pub fn verify_signature(
    pubkey: &[u8],
    message: &[u8],
    signature: &[u8]
) -> bool {
    // WASM implementation
}
```

Network (`network/mod.rs`)

- Peer serialization
- Message encoding/decoding
- Gossip framing

Verification (`verification/mod.rs`)

- SHA-256 hashing (100x faster)
- Signature validation (Ed25519, ECDSA)
- Merkle proof checking

Governance (`governance/mod.rs`)

- Validator set management
- Stake calculations
- Voting logic

Emergency (`emergency/mod.rs`)

- Rollback procedures
- State recovery
- Halt/resume

Kernel (`kernel/`)

- Transition execution
- State encoding
- Invariant checking

Performance:

Operation	JS	WASM	Speedup
SHA-256 (1KB)	5ms	0.05ms	100x
Signature verify	20ms	2ms	10x
JSON serialization	1ms	0.1ms	10x
Full consensus round	500ms	10ms	50x

FFI Bridge:

```typescript
```

```
// JS side
import init, { verify_signature, sha256 } from './wasm-kernel.js';

await init();

const isValid = verify_signature(pubkeyBytes, msgBytes, sigBytes);
const hash = sha256(dataBytes);
```

API:

```rust
#[wasm_bindgen]
pub fn kernel_apply(state: &[u8], event: &[u8]) -> Result<Vec<u8>, String>;

#[wasm_bindgen]
pub fn kernel_verify(state: &[u8], proof: &[u8]) -> Result<bool, String>;

#[wasm_bindgen]
pub fn sha256(data: &[u8]) -> Vec<u8>;

#[wasm_bindgen]
pub fn verify_signature(pubkey: &[u8], msg: &[u8], sig: &[u8]) -> bool;
```

Constraints:

- WASM module < 5MB
- No external dependencies (pure Rust)
- No panic! (use Result)
- Memory-safe (Rust compiler guarantees)

Data Model

Global State (`AppState`)

```typescript
```

```typescript
interface AppState {
  // Model selection
  model: string | null;

  // Streaming
  streaming: boolean;
  ollamaOk: boolean;

  // Messages
  messages: Message[];
  conversations: Record<string, Conversation>;

  // Kernel mode
  kernelMode: boolean;
  kernelRunning: boolean;
  kernelViews: KernelView[];
  kernelRuns: KernelRun[];

  // Flow/compile
  flowStage: number;
  flowModules: FlowModule[];
  compiledFlows: CompiledFlow[];

  // Network
  peers: PeerInfo[];
  ledgerHeight: number;

  // Metadata
  memoryStats: MemoryStats;
  lastEventSeq: number;
  lastEventHash: string;
}

interface Message {
  role: 'user' | 'assistant';
  text: string;
  timestamp: number;
  tokens?: number;
}
```

```typescript
interface Conversation {
  id: string;
  title: string;
  messages: Message[];
  created: number;
  updated: number;
}

interface KernelView {
  id: string;
  type: string;
  data: any;
  hash: string;
}

interface MemoryStats {
  total: number;
  messages: number;
  conversations: number;
  peakMemory: number;
}
```

Consensus Protocol

Finality Mechanism

Definition: An event is *final* when it has been:

1. Verified (I1–I6)
2. Ordered (canonical position)
3. Executed (state transition)
4. Acknowledged by >2/3 of peers

Timeline:

```
T0: Emit event locally
T1: Peer 1 acknowledges
T2: Peer 2 acknowledges
T3: Peer 3 acknowledges (quorum reached: 3/4 > 2/3)
    → Event FINAL (irreversible)
T4: Peer 4 acknowledges (late)
```

Byzantine Safety:

- Theorem: If <1/3 peers are Byzantine, no two conflicting events finalize
- Proof: Two conflicting events E1 and E2 can each get >2/3 acks only if they share common validators. Shared validators signed both → slashable offense → economically irrational.

Liveness:

- Theorem: If <1/3 peers are Byzantine, all valid events eventually finalize
- Proof: Honest peers > 2/3 can form quorum without Byzantine participation.

Determinism Guarantees

Theorem 1: Convergence

Statement: All peers with same event log derive identical state.

Proof:

1. `deriveState()` is a pure function (no side effects)
2. Pure functions: `f(x) = f(x)` (same input → same output)
3. Same eventLog → same deriveState(eventLog) → same state
4. QED ✓

Theorem 2: Auditability

Statement: Every state value can be traced to the event that caused it.

Proof:

1. State is computed from eventLog only

2. Each event is hashed and immutable
3. Hash chain proves no tampering
4. Audit trail = eventLog (no separate record needed)
5. QED ✓

Theorem 3: Replay

Statement: Replaying from genesis produces identical final state.

Proof:

1. Let S0 = genesis state
2. Let T0 = transition after E0: S1 = deriveState([E0])
3. Let T1 = transition after E1: S2 = deriveState([E0, E1])
4. ...
5. Let Tn = transition after En: Sn = deriveState([E0...En])
6. Replaying all events: final_state = deriveState([E0...En]) = Sn
7. QED ✓

Theorem 4: Hash Chain Integrity

Statement: Modifying any event invalidates all downstream hashes.

Proof:

1. T_i = hash(T_{i-1}, E_i, S_i)
2. If E_i is modified → S_i changes → hash changes → T_i changes
3. If T_i changes → hash(T_{i+1}) no longer matches → detected
4. Corruption cannot be hidden
5. QED ✓

API Reference

L1: Surface API

```javascript
```

```javascript
// Emit user action
sovereignLog.emit({
  type: 'MESSAGE_ADDED',
  text: 'Hello, world!',
  timestamp: Date.now()
});

// Subscribe to state changes
sovereignLog.subscribe((state, record) => {
  console.log('State changed:', state);
  console.log('By event:', record.type);
});

// Get current state
const state = sovereignLog.getState();

// Get historical state
const oldState = sovereignLog.deriveState(
  sovereignLog.getLog().slice(0, 50)
);
```

L3: Log API

```javascript
```

```
import {
  emit,
  deriveState,
  snapshot,
  replay,
  restore,
  getLog,
  subscribe,
  snapshot,
  memoryTimeline,
  compiledFlows,
  slice
} from './sovereign-log.js';

// Emit event
const record = emit({
  type: 'MODEL_SELECTED',
  model: 'llama3'
});

// Get state at specific point
const state = deriveState(getLog().slice(0, 100));

// Create checkpoint
const snap = snapshot();
localStorage.setItem('checkpoint', JSON.stringify(snap));

// Recover from checkpoint
const saved = JSON.parse(localStorage.getItem('checkpoint'));
const recovered = replay(saved.log);

// Restore live
const state = restore(saved.log);

// Query timeline
const timeline = memoryTimeline();
timeline.forEach(e => console.log(e.type, e.count, e.seq));

// Query flows
const flows = compiledFlows();
```

```
flows.forEach(f => console.log(f.moduleName, f.ir));

// Query range
const range = slice(50, 100);
```

L4: Network API

```javascript
```

```javascript
import {
  addPeer,
  broadcastEvent,
  getAcknowledgments,
  isEventFinal,
  getLedger,
  getPeers
} from './sovereign-network.js';

// Connect to peer
addPeer('peer-123', pubkey);

// Broadcast event to network
broadcastEvent(eventRecord);

// Check finality
if (isEventFinal(eventHash)) {
  console.log('Event is final');
}

// Get acks
const acks = getAcknowledgments(eventHash);
console.log(`${acks.length} peers acknowledged`);

// View ledger
const ledger = getLedger();
ledger.forEach(entry => {
  console.log(entry.type, entry.event.type);
});

// View peers
const peers = getPeers();
peers.forEach(p => console.log(p.peerId, p.reputation));
```

L5: Kernel API

```javascript
```

```
import {
  verifyEvent,
  canonicalizeEventBatch,
  createTransitionRecord,
  verifyTransitionChain,
  createSnapshot,
  verifySnapshot
} from './rekernel.js';

// Verify event
const result = verifyEvent(event);
if (!result.valid) {
  console.log('Violations:', result.violations);
}

// Order events
const ordered = canonicalizeEventBatch([e1, e2, e3]);

// Create transition
const transition = await createTransitionRecord({
  index: 42,
  prevTransitionHash: lastHash,
  event: event,
  preStateHash: oldState,
  postStateHash: newState
});

// Verify chain
if (verifyTransitionChain(chain)) {
  console.log('Chain integrity verified');
}

// Create snapshot
const snap = await createSnapshot(state, 1000);
console.log('Snapshot at height', snap.height);

// Verify snapshot
if (verifySnapshot(snap)) {
```

```
  console.log('Snapshot is valid');
}
```

L6: Compute API

```javascript
```

```javascript
import {
  JSONFlowExecutor,
  programHash,
  validateModule
} from './udcsef.js';

// Create executor
const executor = new JSONFlowExecutor();

// Register ops
executor.registerOp('double', ({ x }) => x * 2);
executor.registerOp('concat', ({ a, b }) => a + b);

// Execute
const result = executor.execute(ir, {
  text: 'hello',
  count: 5
});

if (result.success) {
  console.log('Output:', result.outputs);
  console.log('Trace:', result.trace);
} else {
  console.log('Error:', result.error);
}

// Content address
const hash = programHash(ir);
console.log('Program ID:', hash);

// Validate
const errors = validateModule(ir);
if (errors.length > 0) {
  console.log('Validation errors:', errors);
}
```

L7: WASM API

```javascript
```

```
import init, {
  verify_signature,
  sha256,
  kernel_apply,
  kernel_verify
} from './wasm-kernel.js';

await init();

// Verify signature
const isValid = verify_signature(pubkey, message, signature);

// Hash
const hash = sha256(data);
const hashHex = Array.from(hash)
  .map(b => b.toString(16).padStart(2, '0'))
  .join('');

// Apply kernel
const newState = kernel_apply(oldState, event);

// Verify proof
const isProofValid = kernel_verify(state, proof);
```

Deployment Guide

Single Node (Development)

```bash
# Start server
node server.js

# Access portal
open http://localhost:3000
```

Multi-Node (Network)

```bash
# Node A
node server.js 3000

# Node B (in different terminal/machine)
node server.js 3001

# In Portal, add peers manually
sovereignNetwork.addPeer('peer-B-id', pubkey);
```

Docker

```dockerfile
FROM node:18-alpine

WORKDIR /app
COPY . .

EXPOSE 3000
CMD ["node", "server.js"]
```

```bash
docker build -t i-am-ios .
docker run -p 3000:3000 i-am-ios
```

Persistence

IndexedDB (automatic):

- Events persisted automatically on every emit
- Survives page reload
- Data stored in browser's IndexedDB quota (~50MB)

Export/Import:

```javascript
// Export ledger
const ledger = sovereignLog.getLog();
const json = JSON.stringify(ledger);
const blob = new Blob([json], { type: 'application/json' });
downloadBlob(blob, 'ledger.json');

// Import ledger
const file = await selectFile();
const json = await file.text();
const ledger = JSON.parse(json);
const state = sovereignLog.restore(ledger);
```

Testing

Unit Tests

```javascript
```

```javascript
import { deriveState } from './sovereign-log.js';

describe('deriveState', () => {
  it('should be idempotent', () => {
    const log = [
      { type: 'MODEL_SELECTED', model: 'llama3', ... },
      { type: 'MESSAGE_ADDED', text: 'hello', ... }
    ];

    const state1 = deriveState(log);
    const state2 = deriveState(log);

    assert.deepEqual(state1, state2);
  });

  it('should reconstruct state from events', () => {
    const log = [
      { type: 'MODEL_SELECTED', model: 'llama3', ... }
    ];

    const state = deriveState(log);
    assert.equal(state.model, 'llama3');
  });
});
```

Integration Tests

```javascript
```

```
describe('Sovereign Stack', () => {
  it('should sync across tabs', async () => {
    // Open portal in Tab A
    const tabA = new Browser('http://localhost:3000');

    // Open portal in Tab B
    const tabB = new Browser('http://localhost:3000');

    // Tab A emits event
    await tabA.emit({ type: 'MESSAGE_ADDED', text: 'hello' });

    // Tab B should receive
    await wait(100);
    const stateB = await tabB.getState();
    assert.equal(stateB.messages.length, 1);
  });

  it('should finalize events across network', async () => {
    // Start Node A and B
    const nodeA = spawn('node', ['server.js', '3000']);
    const nodeB = spawn('node', ['server.js', '3001']);

    // Connect peers
    // Emit event on A
    // Wait for quorum
    // Verify finality on B
  });
});
```

Appendices

A. Hash Chain Example

```
Event 0: { type: 'START', seq: 0 }
  payload = { type: 'START', seq: 0, ts: 1000 }
  hash = fnv32(payload | '0000000000000000')
       = 'a1b2c3d4'
```

```
Event 1: { type: 'MESSAGE', text: 'hello', seq: 1 }
  payload = { type: 'MESSAGE', text: 'hello', seq: 1, ts: 2000 }
  hash = fnv32(payload | 'a1b2c3d4')
       = 'e5f6g7h8'

Event 2: { type: 'MESSAGE', text: 'world', seq: 2 }
  payload = { type: 'MESSAGE', text: 'world', seq: 2, ts: 3000 }
  hash = fnv32(payload | 'e5f6g7h8')
       = 'i9j0k1l2'

Chain: a1b2c3d4 → e5f6g7h8 → i9j0k1l2 → ...

If Event 1 text changed to 'goodbye':
  new_payload = { type: 'MESSAGE', text: 'goodbye', seq: 1, ts: 2000 }
  new_hash = fnv32(new_payload | 'a1b2c3d4')
           = 'xyz123'  (different!)

  This invalidates hash of Event 2:
  hash_of_2 = fnv32(payload_2 | 'xyz123')
            ≠ 'i9j0k1l2'

  Corruption DETECTED ✓
```

B. JSONFlow Example: Word Count

```
json
```

```json
{
  "module_name": "word_counter",
  "version": "1.0.0",
  "inputs": [
    { "name": "text", "type": "string" }
  ],
  "outputs": [
    { "name": "wordCount", "type": "number" },
    { "name": "words", "type": "array" }
  ],
  "steps": [
    {
      "id": "split",
      "op": "string_split",
      "inputs": { "text": "$text", "sep": " " },
      "outputs": { "result": "$words" }
    },
    {
      "id": "count",
      "op": "array_length",
      "inputs": { "array": "$words" },
      "outputs": { "result": "$wordCount" }
    }
  ]
}
```

Execution:

```
Input: { text: "hello world test" }

Step 1 (split):
  - input: { text: "hello world test", sep: " " }
  - op: string_split
  - output: { result: ["hello", "world", "test"] }
  - env: { text: "hello world test", words: ["hello", "world", "test"] }

Step 2 (count):
  - input: { array: ["hello", "world", "test"] }
  - op: array_length
  - output: { result: 3 }
  - env: { ..., wordCount: 3 }

Final output: { wordCount: 3, words: ["hello", "world", "test"] }
```

C. Consensus Example (5 Peers)

```
Quorum needed: ceil(2×5/3 + 1) = ceil(4.33) = 5

Scenario: All honest (0 Byzantine)

Peer A: broadcasts event E
Peer B: receives, verifies, sends ack
Peer C: receives, verifies, sends ack
Peer D: receives, verifies, sends ack
Peer E: receives, verifies, sends ack

Acks received: 5 (100%) → finality threshold crossed
Event is FINAL ✓

Byzantine scenario: 1 dishonest peer

Peer A: broadcasts event E
Peer B: receives, sends ack
Peer C: receives, sends ack
Peer D: receives, sends ack
Peer E (Byzantine): ignores or sends ack
```

```
Acks needed: 5, but max can get: 4 (from honest peers)
Scenario fails → no finality

For finality with 1 Byzantine:
Need 2/3 quorum from (N - 1) honest peers
i.e., need acks from 4 peers out of 5 total
Honest peers can form 4-peer quorum → finality possible ✓
```

D. Invariants Checklist

- [ ] All events hashed and immutable
- [ ] deriveState is pure (no side effects)
- [ ] Hash chain unbroken (all Ti = hash(Ti-1, Ei, Si))
- [ ] All events verified before execution (I1–I6)
- [ ] All peers have same event log (P2P sync complete)
- [ ] Finality requires >2/3 quorum acknowledgment
- [ ] State derived from log (not separate DB)
- [ ] All peers derive identical state (convergence)
- [ ] Replaying from genesis produces same final state (auditability)
- [ ] Modifying any event invalidates all downstream hashes (tamper-proof)

Version History

Version	Date	Changes
2.0	2024-04-13	Full specification, time-machine model, WASM integration
1.5	2024-01-15	Consensus protocol formalized
1.0	2023-11-01	Initial architecture

References

- Sovereign Log Implementation
- Rekernel Architecture

- Consensus Protocol
- WASM Kernel
- JSONFlow Executor

License

Sovereign OS Community License

I-AM-IOS Specification © 2024

"In the beginning, there was an event. From that event came state. From state came consensus. And from consensus came the future."